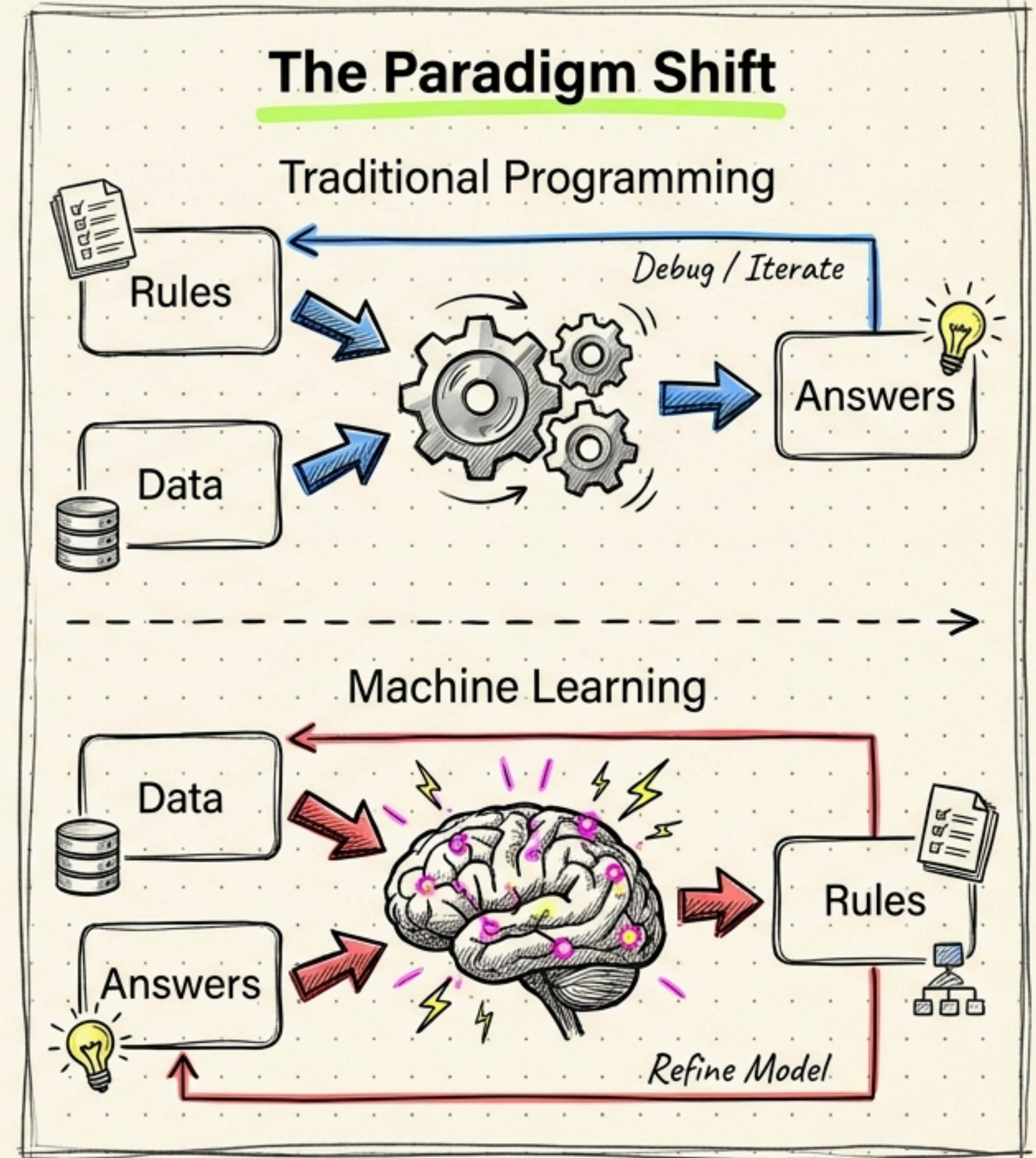


Machine Learning

Teaching a computer to **learns from experience** — **without writing rules** for every situation.

Think of a child learning to recognize cats. You don't hand them a rulebook saying, "if it has whiskers and four legs, it's a cat." You just show them 1000 photos of cats and non-cats. They figure it out themselves. That's exactly how this works.

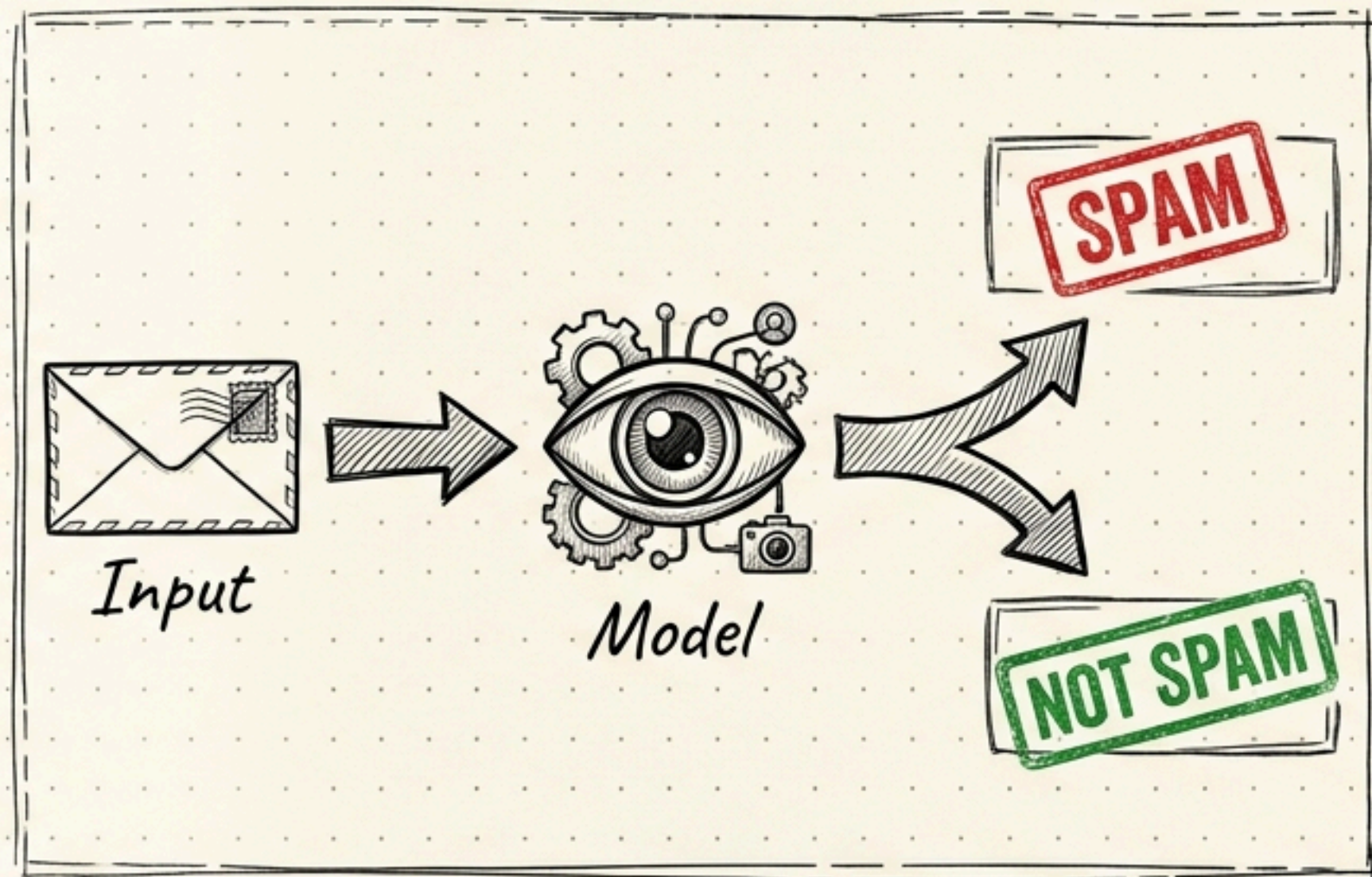
Fun Fact! 🤖 Netflix uses ML to learn your taste—just like the child learning cats.



Supervised Learning

Teaching a model using **labeled examples** — where every input has a correct answer already attached.

Imagine a teacher giving students a practice exam with the answer key attached. The student studies the questions AND the answers together. The model learns from examples where the answer is already known.

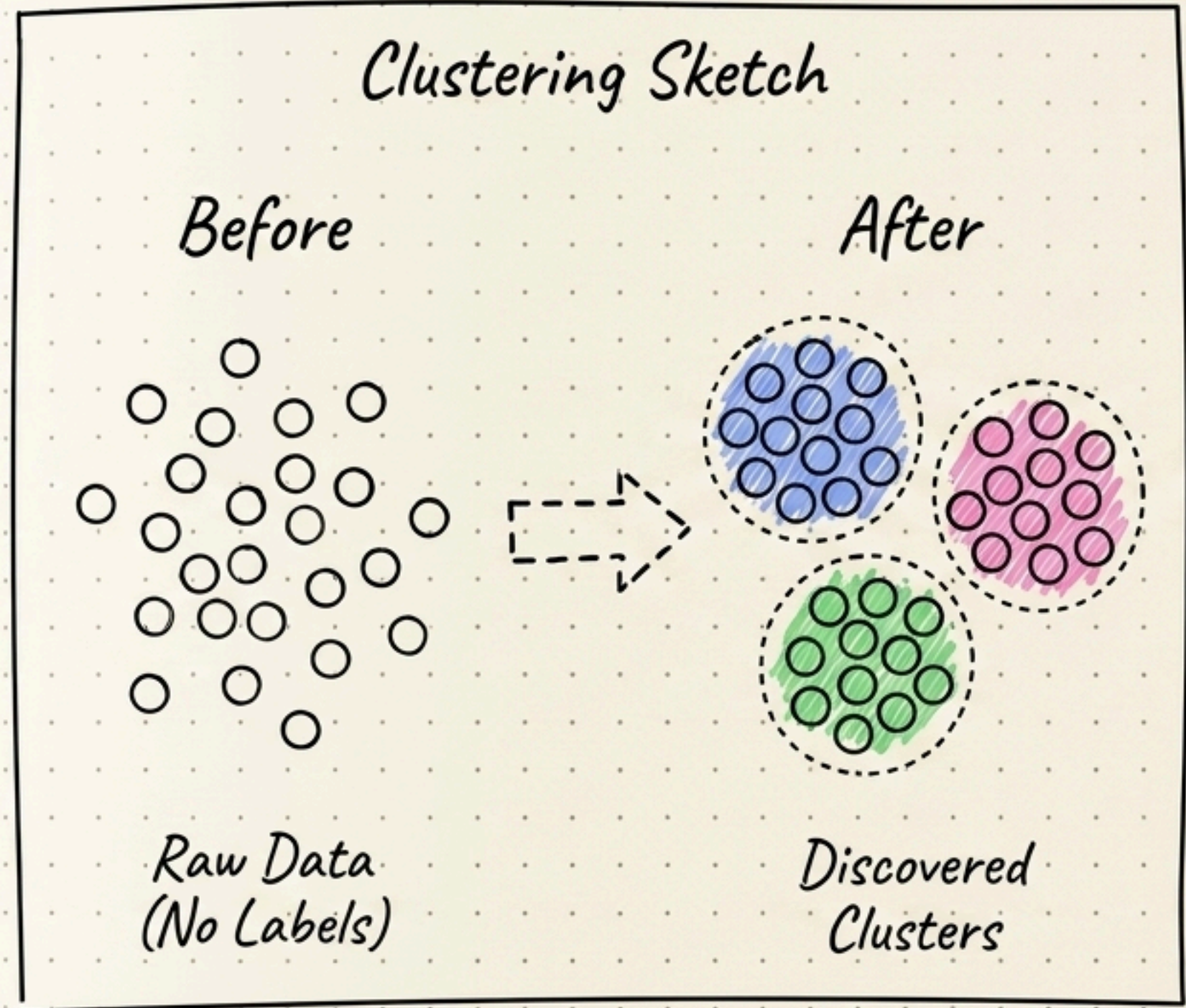


★ Math translation: $\text{Input } X \rightarrow \text{Model } f(x) \rightarrow \text{Output } Y$ (Y is already known!)

Unsupervised Learning

Finding hidden patterns in data — without any labels or correct answers given.

Imagine dropping a new student into a school canteen with no seating chart. They naturally observe and group themselves — sports kids here, art kids there, science nerds over there. Nobody told them who was who; they figured it out from natural patterns.



Reinforcement Learning

An agent learns by taking actions, receiving rewards or penalties, and improving over time.

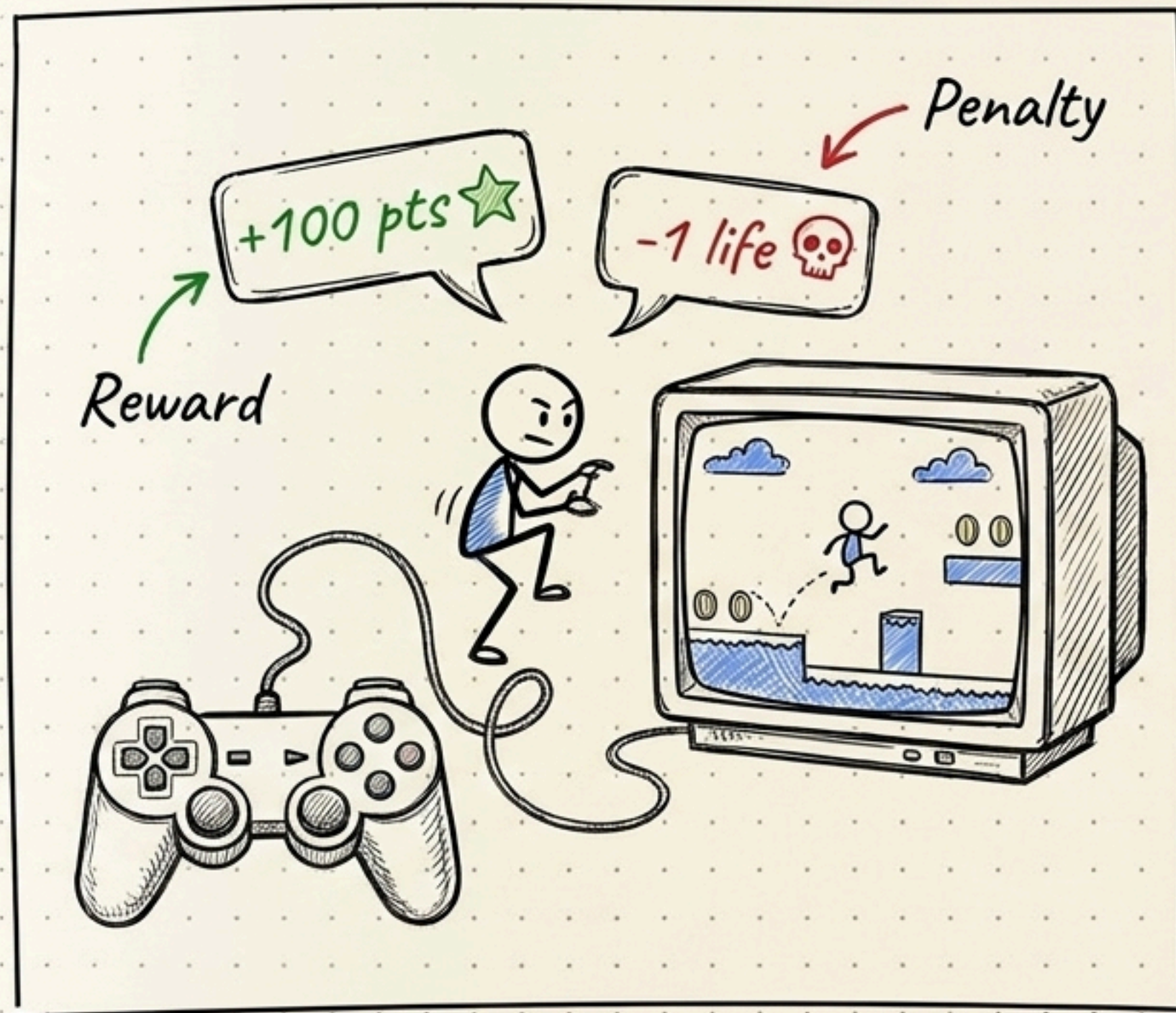
It's just like training a dog.

Dog sits → you give it a treat (reward) 🍖

Dog jumps on a guest → you say "No" (penalty) 😞

Over time, the dog learns exactly what gets treats. We do the same thing with machines.

Goal = Maximize Total Reward! (Fun fact: This is how they trained ChatGPT!)



Overfitting and Underfitting

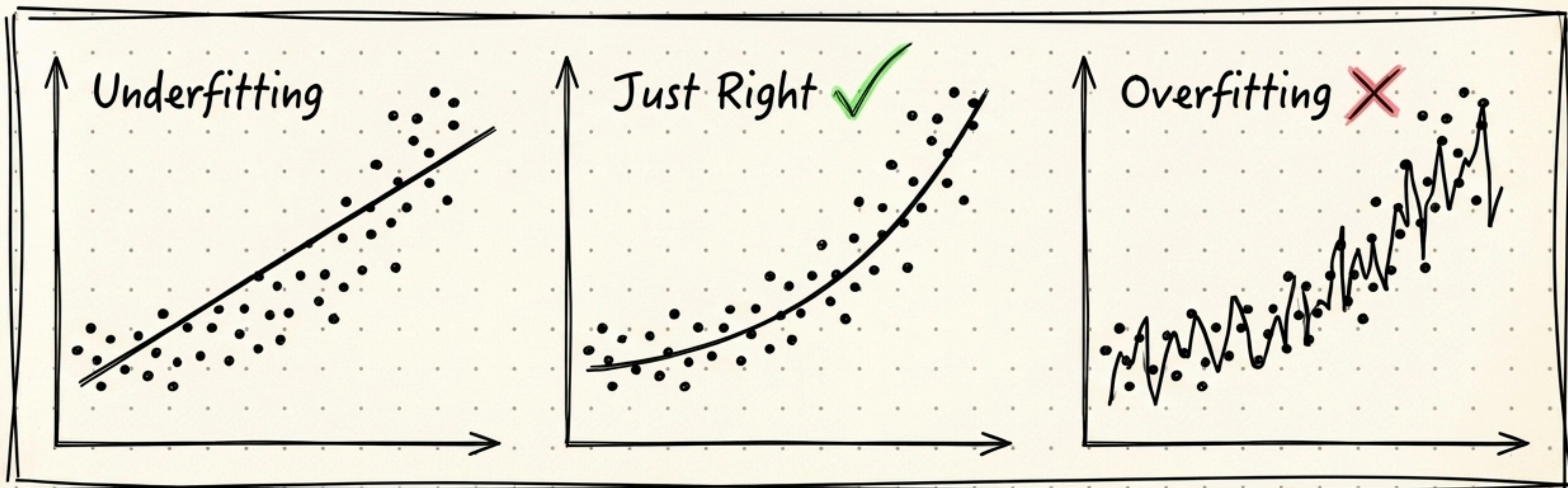
Overfitting = memorizing the training data too well.

Underfitting = too simple, didn't learn enough.

📖 **UNDERFITTING:**
You only read chapter headings and failed.

📖 **JUST RIGHT:**
You understood the core concepts.

📖 **OVERFITTING:**
You memorized ONE sample paper. You got 100% on practice, but failed the real exam.



Bias and Variance Tradeoff

Bias = error from wrong assumptions.

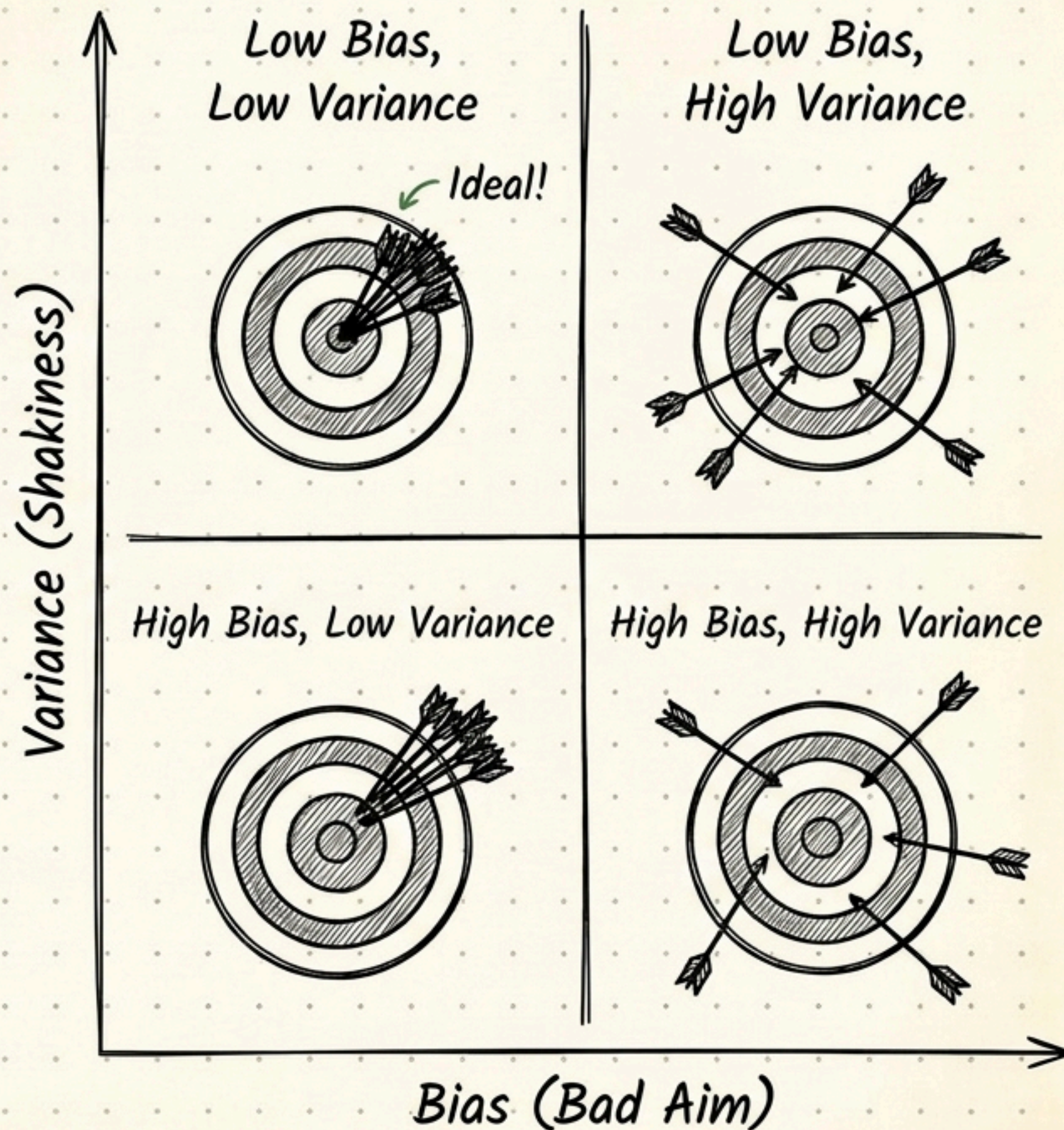
Variance = error from being too sensitive to training data. The goal is the sweet spot.

Think of an archery target.

High Bias means your bow is aimed wrong—you consistently hit the same wrong spot.

High Variance means you have shaky hands—your arrows are scattered everywhere.

We want steady hands and good aim.

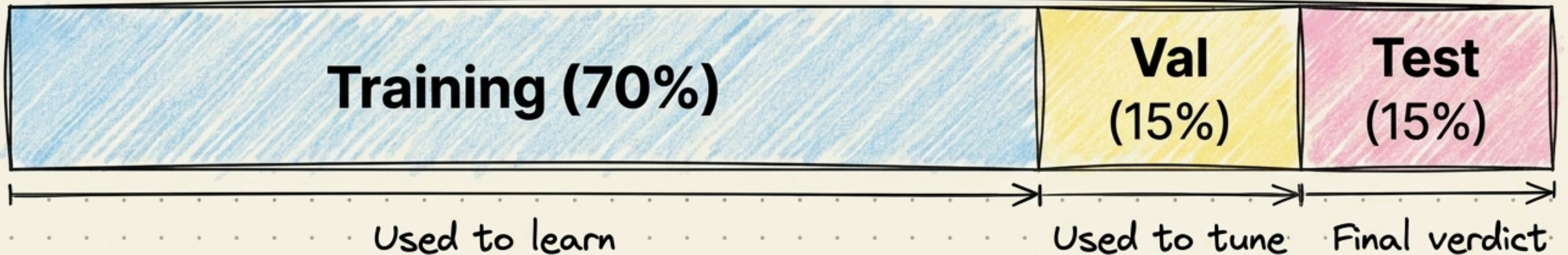


Training, Validation and Test Set

Three separate portions of data to **teach, tune, and evaluate** the final model.

Back to the student prepping for board exams:

- 📚 **TRAINING SET:** Your textbooks and practice problems.
- 📝 **VALIDATION SET:** The mock tests you take to adjust your study strategy.
- 📋 **TEST SET:** The actual final board exam, seen for the very first time.



⚠️ **NEVER** let the model see the Test Set during training! It's a blind exam.

Features and Labels

Features are the **input clues** given to the model. **Labels** are the **correct answers** it's trying to predict.

If you're predicting a house's price, the **FEATURES** are the inputs: number of rooms, square footage, neighborhood, and age. The **LABEL** is the final output: the actual price tag of the house.

FEATURES (X)				LABEL (Y)
Rooms	Location	Size	Age	Price
3	Suburban	1800 sq ft	15 yrs	\$400k
4	Urban	2200 sq ft	10 yrs	\$250k
5	Urban	2200 sq ft	15 yrs	\$550k
6	Urban	1800 sq ft	12 yrs	\$320k
3	Suburban	2200 sq ft	10 yrs	\$450k
5	Urban	1900 sq ft	15 yrs	\$550k

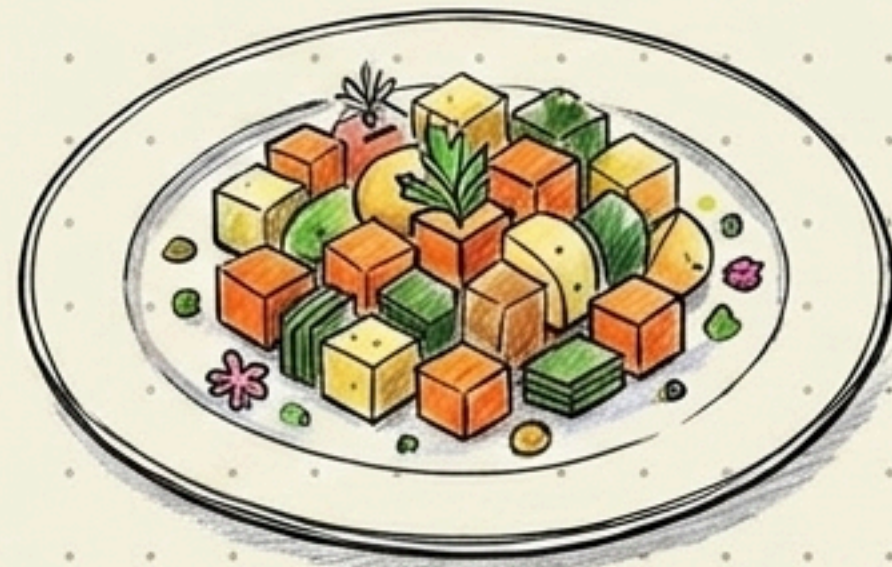
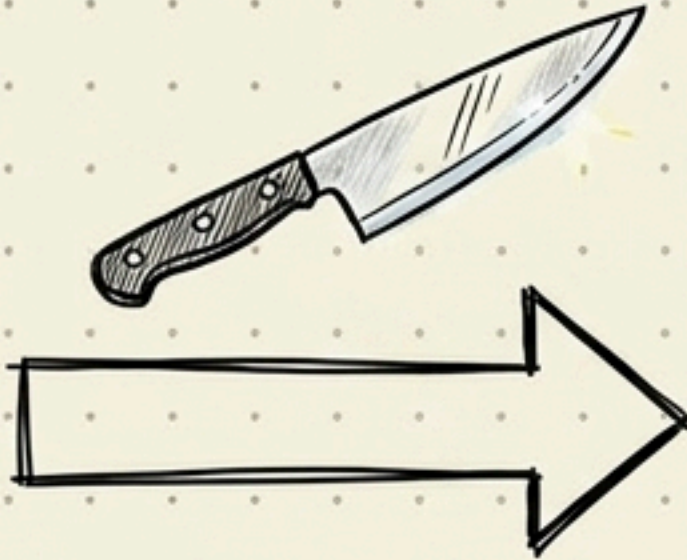
Feature Engineering

The art of creating, **transforming**, or **selecting** the best input features to **improve performance**.

Think of a chef preparing ingredients. You don't throw whole, unwashed, raw vegetables into a pot. You chop, marinate, season, and prep them perfectly. Feature engineering is prepping your raw data to be the best possible dish for your ML model.



Raw Data



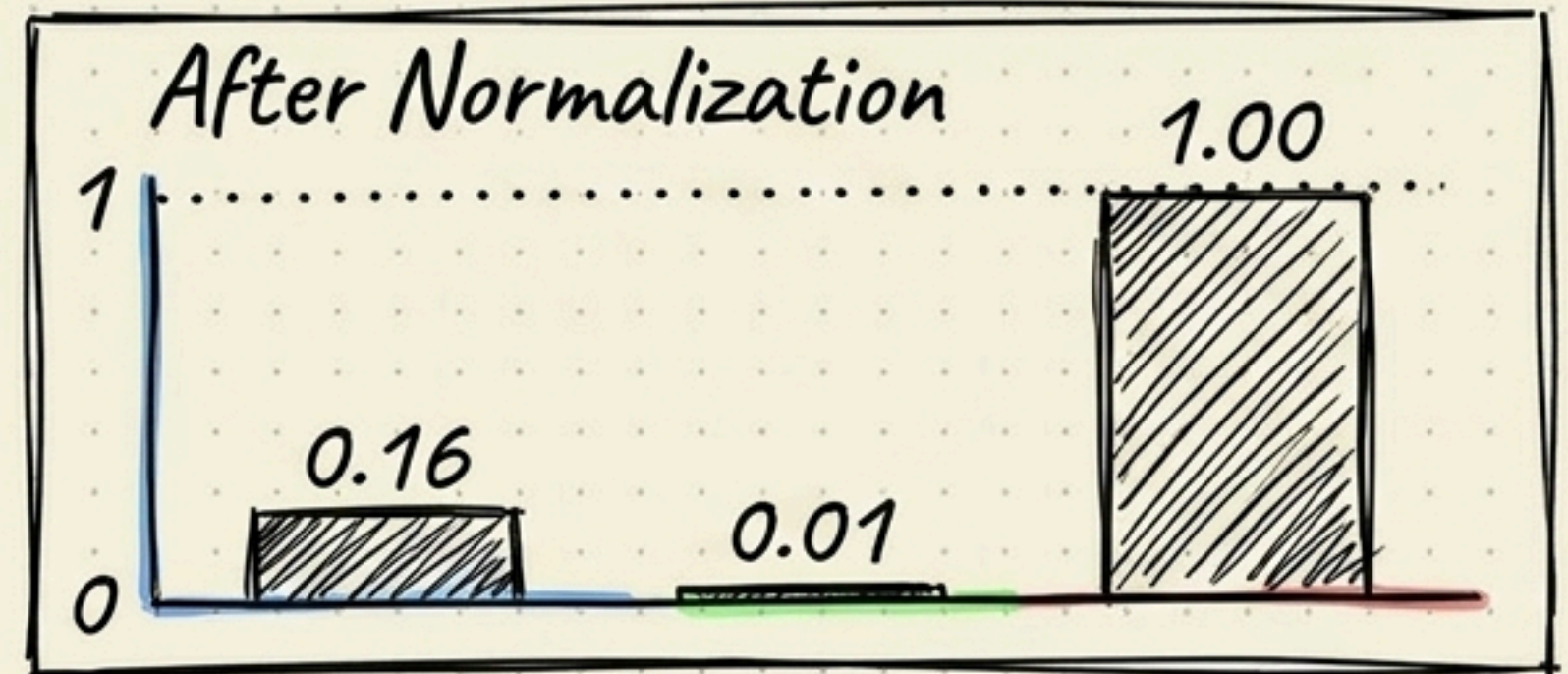
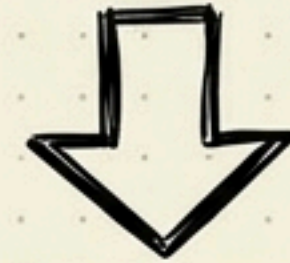
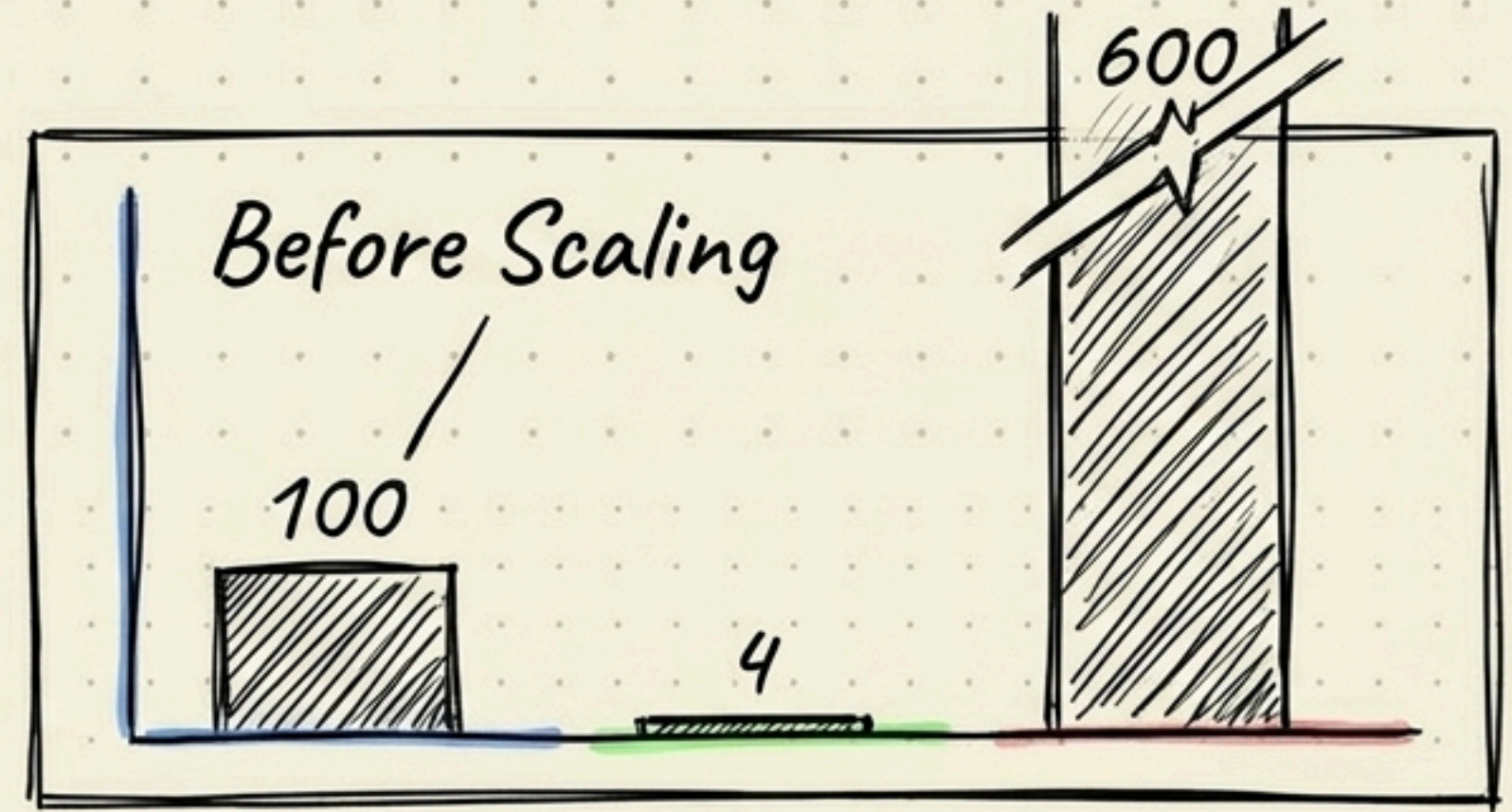
Engineered Features

*Industry Reality:
70-80% of a Data
Scientist's time is
spent right here!*

Normalization and Standardization

Scaling techniques to bring all feature values into a **comparable range** so **nothing dominates**.

*Imagine comparing students globally.
One student has a 95 (percentage),
another has a 3.8 (GPA), and another
has a 540 (total marks).
You can't compare them directly!
You have to convert them all to the
same scale.*

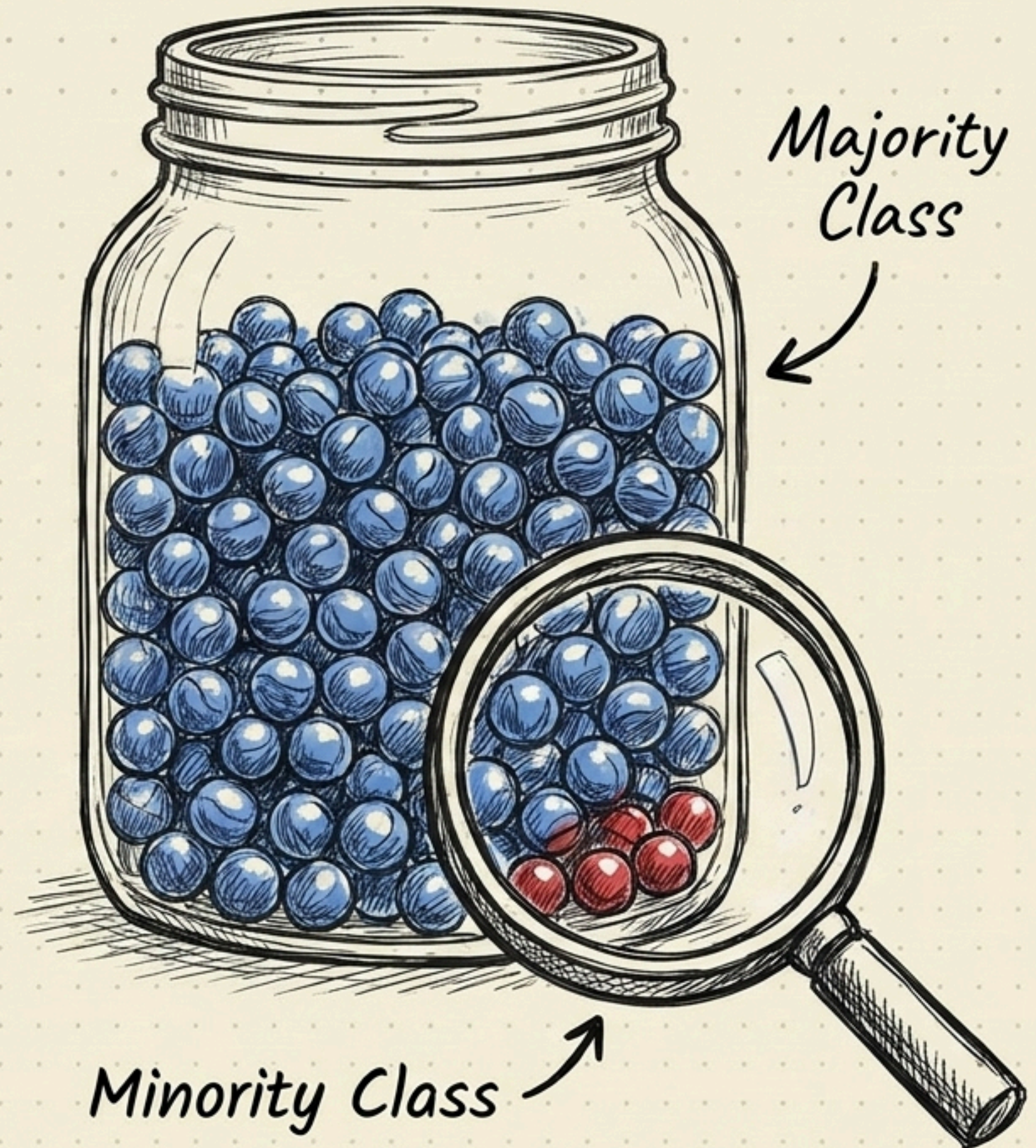


Imbalanced Dataset

When one class appears **far more frequently** than another, **biasing the model**.

Imagine a class of 100 students: 95 are right-handed, 5 are left-handed. If you build a model to guess handedness, it could just lazily yell "Right-handed!" every time. It would technically be 95% accurate—but it would completely fail to ever find a left-handed person.

Super common in Fraud Detection (99.9% normal, 0.1% fraud).



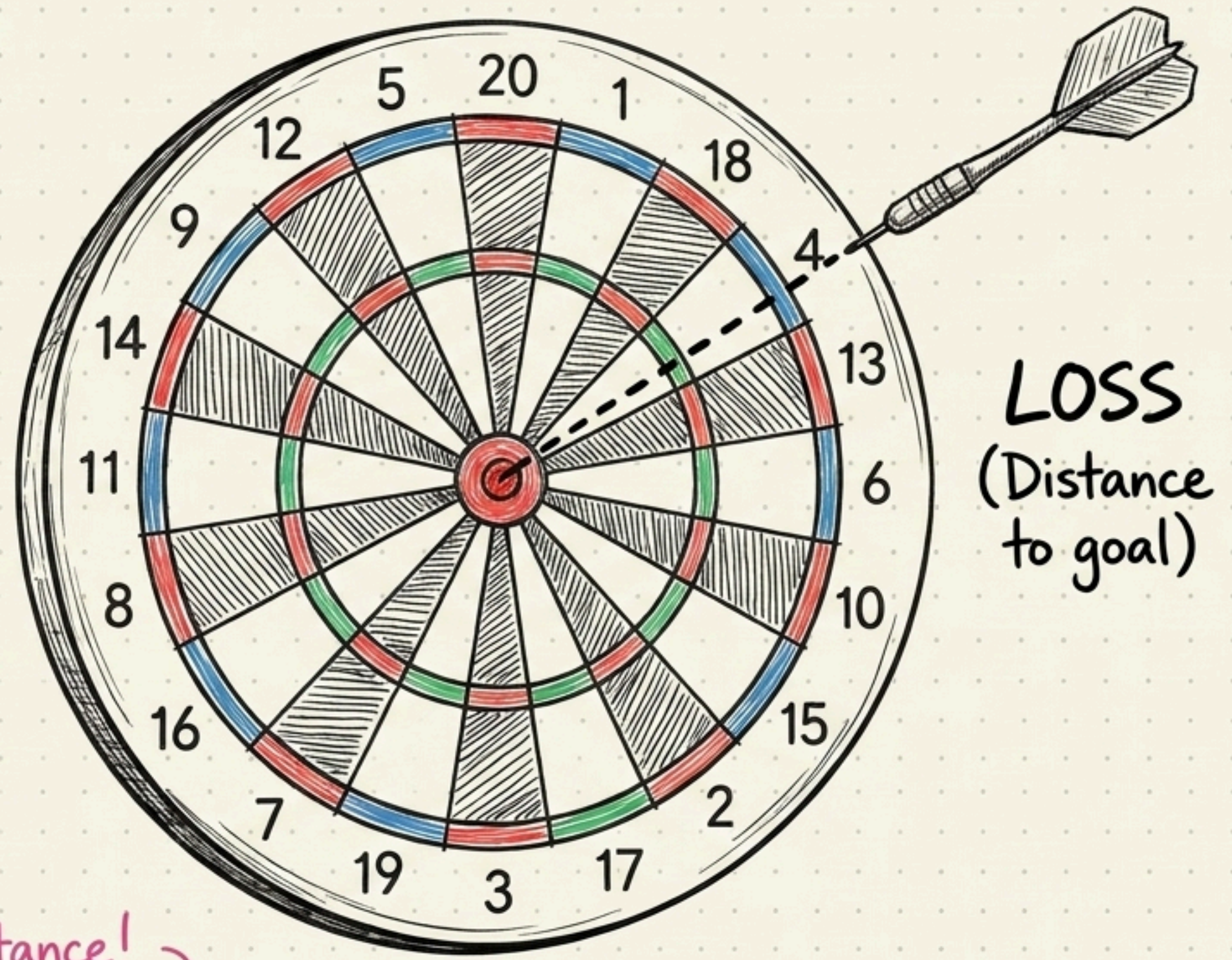
Loss Function

A mathematical formula that measures exactly how wrong the model's predictions are. The model tries to MINIMIZE this.

Think of a GPS navigation system. When you take a wrong turn, the GPS calculates exactly "how far off route" you are and recalculates.

The loss function is the model's GPS—it calculates the distance between the wrong prediction and the correct answer.

Training = continuously shrinking this distance!
$$MSE = (1/n) \times \sum (\text{actual} - \text{predicted})^2$$

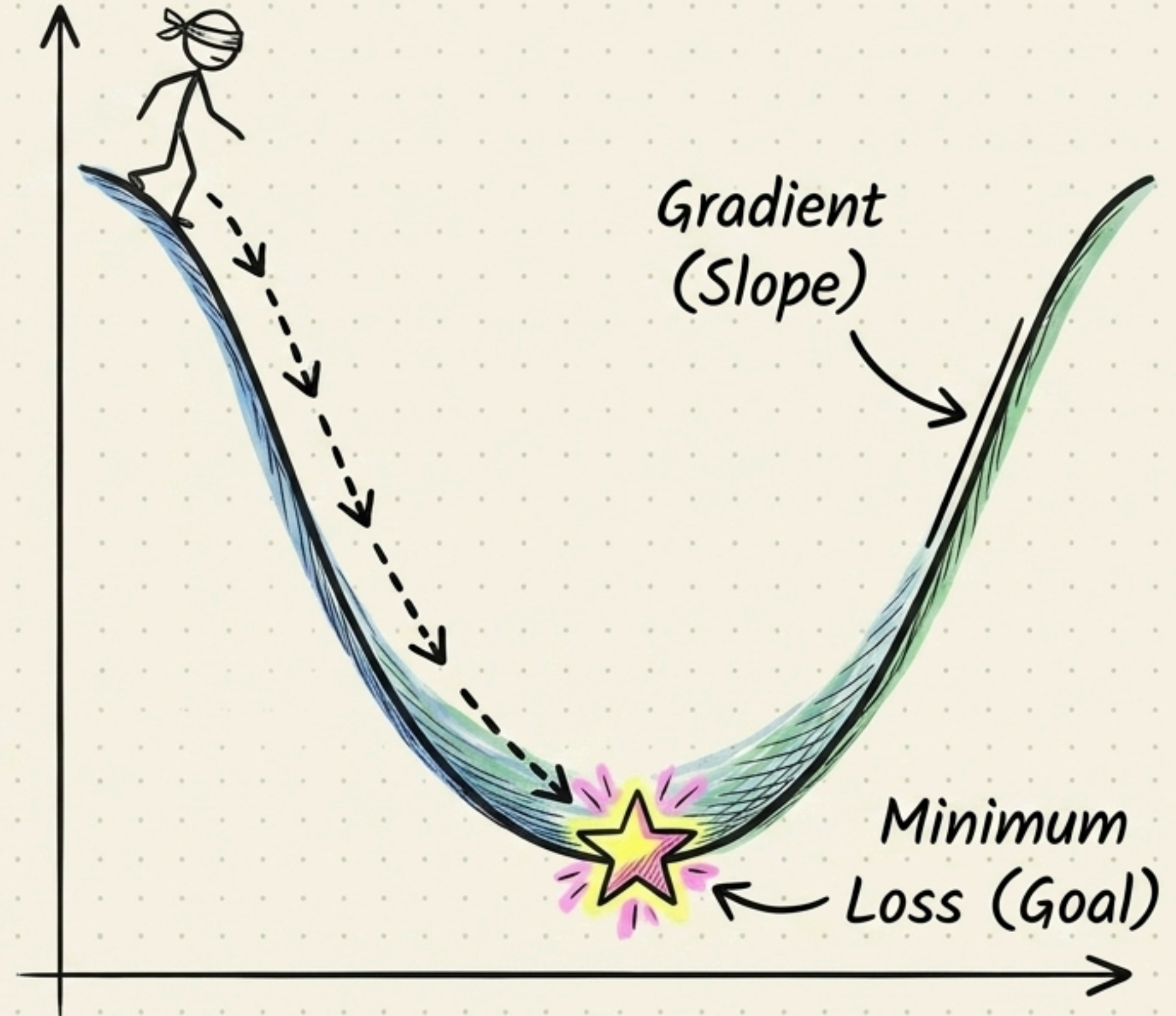


Gradient Descent

An optimization algorithm that helps the model find the **minimum loss step-by-step** by taking steps in the downhill direction.

Imagine you're blindfolded on a hilly mountain and need to reach the lowest valley. You can only feel the ground.

Your strategy? Take a small step in whichever direction feels downhill. Repeat step-by-step until you hit the bottom.

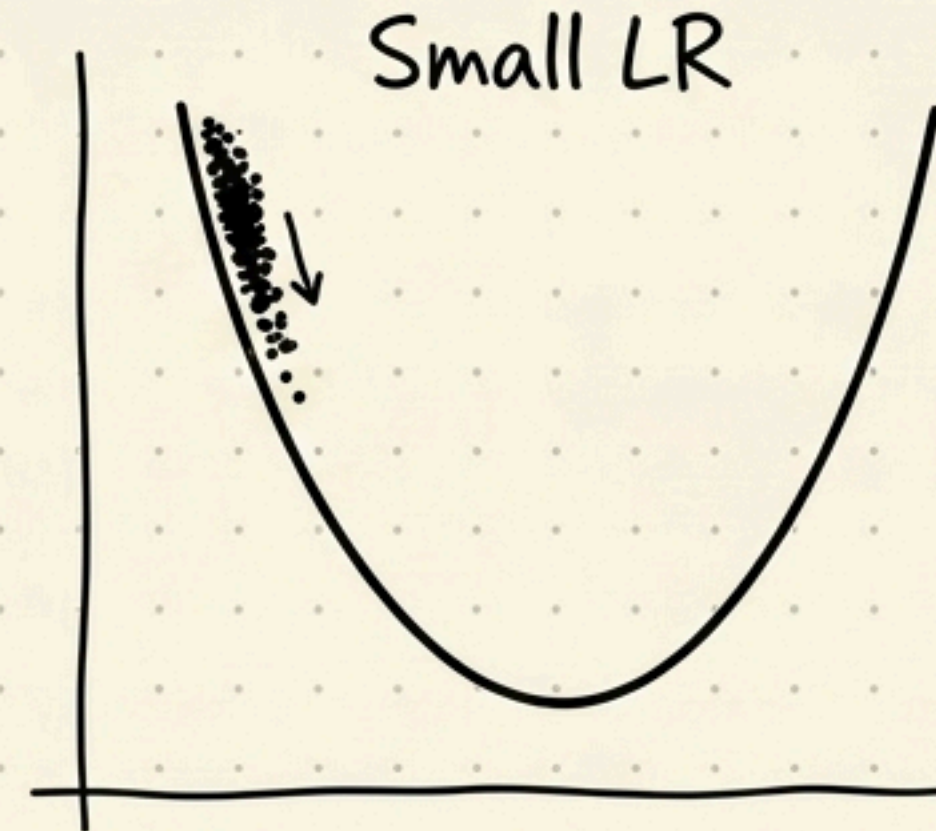
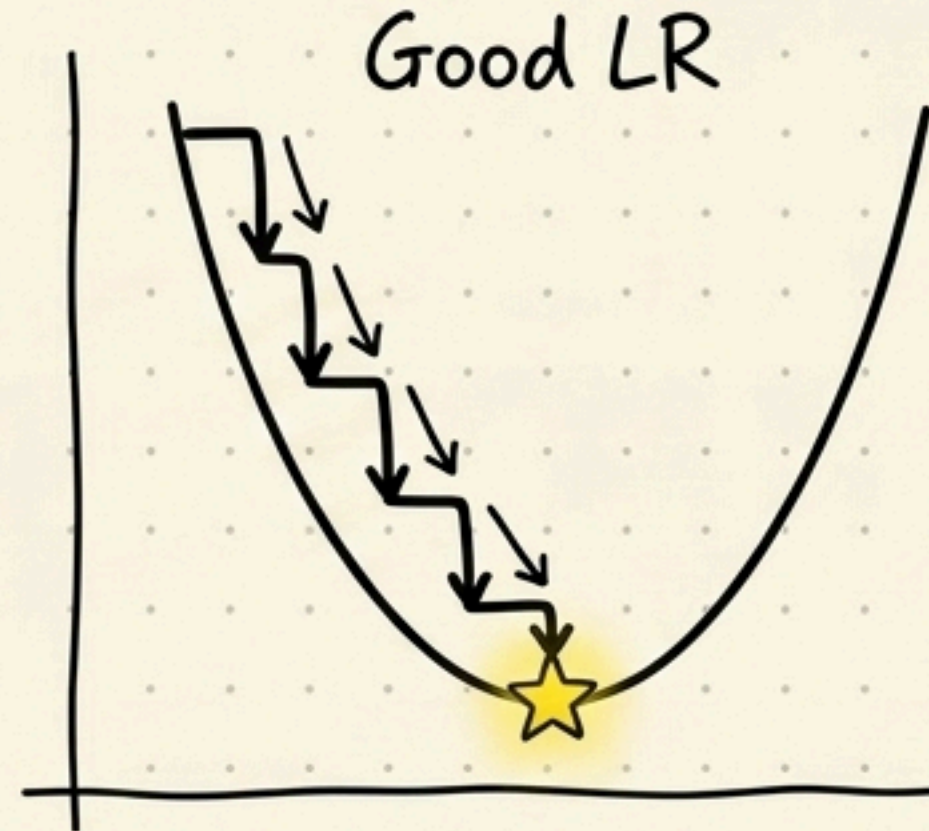
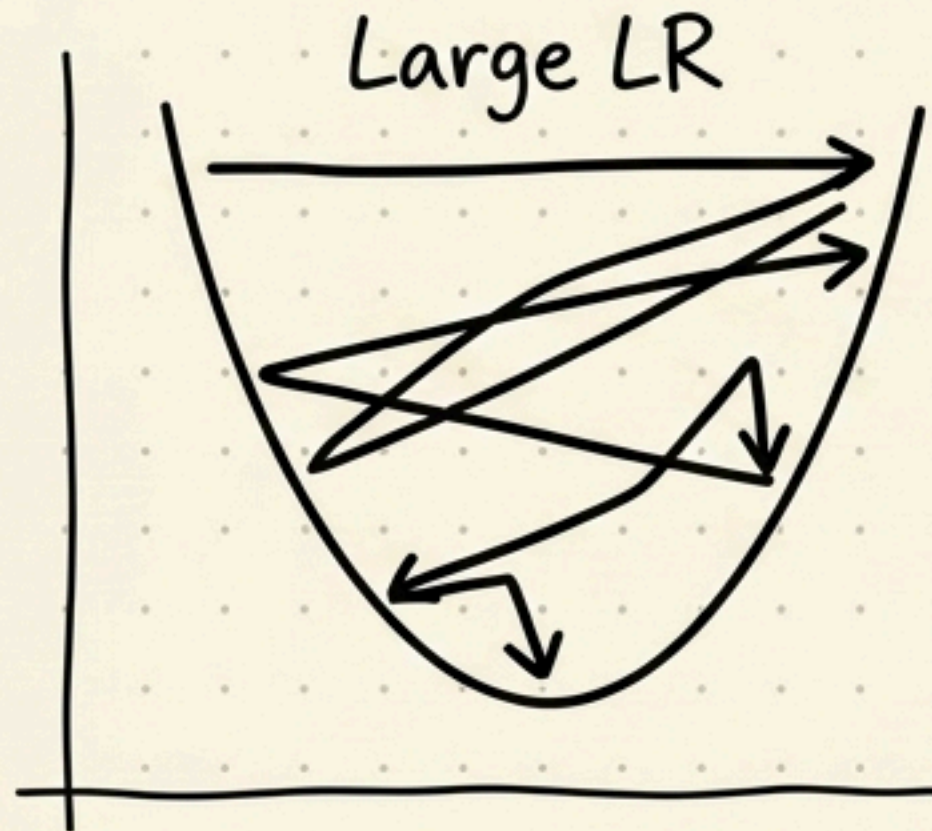


Learning Rate

A setting that **controls exactly how big each step is** during gradient descent.

You're still blindfolded on that mountain.

- 🦶 **TOO LARGE** a step: You jump past the valley, bounce to the other side, and never settle.
- 🐌 **TOO SMALL** a step: You shuffle so slowly it takes forever to reach the bottom.
- ✓ **JUST RIGHT**: Steady, confident steps.



💡
Pro tip:
Start at 0.1,
0.01, or 0.001
and experiment!

Parameters vs Hyperparameters

Definition: Parameters = values the model **learns automatically** from data.
Hyperparameters = settings **YOU set** before training begins.

Analogy: Imagine baking a cake. You decide the oven temperature and pan size before you start—these are your hyperparameters. How the batter actually rises and the crust forms happens automatically during baking—those are the parameters.

